

A Practical Introduction to Cursor and ChatGPT on macOS

Tools, skills, context, and agents

This guide explains how to combine Cursor and the ChatGPT Mac app for programming, research, writing, and technical projects. It focuses on the ideas that determine quality: what the systems can see, what they can do, how reusable instructions work, and how to keep control of the result.

September 2026

Contents

1	The basic idea	2
1.1	A simple example	2
2	Install and prepare the Mac	2
2.1	Cursor	2
2.2	ChatGPT for macOS	3
2.3	What to test immediately	3
3	Model, tools, context, skills, and agents	3
3.1	Tool versus skill	3
3.2	Agent versus ordinary chat	4
4	Context: what the AI can see	4
4.1	The five-part task description	4
4.2	Context in Cursor	5
4.3	Context in ChatGPT	5
4.4	When to start a new chat	5
5	Rules and skills	5
5.1	A practical skill structure	6
6	Agents and modes of work	6
6.1	Cursor Agent	6
6.2	ChatGPT Work and Plan	6
6.3	A safe autonomy ladder	6
7	A reliable combined workflow	7
7.1	Example: fixing a bug	7
7.2	Example: adding a feature	7
7.3	Example: research or writing	8
8	Prompt patterns	8
8.1	Understand before changing	8
8.2	Implement with a boundary	8
8.3	Debug from evidence	8
8.4	Review	8
9	Common mistakes and safety	8
10	One-minute checklist	9
11	Further reading	9

The basic idea

Cursor and ChatGPT are interfaces to language models that can inspect context, use tools, produce artefacts, and sometimes carry out a sequence of actions. They are not automatically aware of the whole problem. The result depends on the context supplied, the tools enabled, and the clarity of the task.

Give the model the right context, the right tools, and a clear definition of success. Ask it to work in small, reviewable steps.

Tool	Best use
ChatGPT for Mac	Explore ideas, explain concepts, research, analyse files, draft specifications, and work beside other applications.
Cursor	Understand and modify a codebase, run commands, fix errors, write tests, refactor, and review changes.
Git	Create recovery points, compare versions, and collaborate safely.

The distinction is not absolute. ChatGPT can work with code and Cursor can discuss architecture. Cursor is organised around a repository and its development tools, while ChatGPT is organised around conversations, projects, files, connected applications, and broader workflows.

A simple example

Suppose you want to add a new data export feature.

1. Ask ChatGPT to help clarify the user need, the possible designs, the risks, and the acceptance criteria.
2. Save the resulting brief as **docs/csv-export.md**.
3. Open the repository in Cursor and ask it to inspect the brief, the existing table component, and the tests.
4. Let Cursor propose a file-level plan before editing.
5. Implement the change in a small branch, run tests, and review the diff.

ChatGPT helps shape the problem. Cursor operates inside the codebase. Git provides a safe history.

Install and prepare the Mac

Cursor

1. Install Cursor from <https://cursor.com/> and sign in.
2. Open the complete project folder, not just one source file. The folder is the unit of context for most coding tasks.
3. If the project uses Git, create a branch before substantial agent changes.
4. Confirm that the project can be run manually and identify its test or build command.
5. Add a short project description and important commands to **AGENTS.md**; Cursor also supports project rules in **.cursor/rules**.

A useful first file might contain:

```
# Project overview
```

```
This is a Python FastAPI service. The API is in app/.
Tests are in tests/. Run them with: pytest -q
Start locally with: uvicorn app.main:app --reload
Do not modify files in data/raw/.
```

ChatGPT for macOS

Install the official app from <https://openai.com/chatgpt/desktop/>, launch it, and sign in. The app must be running for desktop integrations to work.

The fastest entry point is the Chat Bar: press **Option+Space**, or click the ChatGPT menu-bar icon. The shortcut can be changed under **ChatGPT > Settings > Keyboard Shortcut**.

To work beside Cursor, open **Work with Apps**. Enable the relevant application or install the required extension. ChatGPT can use recognised text from the active editor or selection as part of the conversation. When code editing is supported, it proposes a diff that you can inspect and apply.

macOS may request Accessibility permission. Grant only what is needed and review it under **System Settings > Privacy & Security > Accessibility**.

What to test immediately

After installation, test three small actions:

1. Open the Chat Bar and ask a simple question.
2. In Cursor, open a small source file and ask for an explanation without requesting edits.
3. Ask Cursor to identify the project's test command, then verify that command yourself.

This separates setup problems from reasoning problems. If the tool cannot see the file, no improvement in prompting will solve the issue.

Model, tools, context, skills, and agents

Concept	Meaning	Example
Model	The system generating the response.	It proposes a design or predicts the next code change.
Tool	An operation available to the model.	Read a file, search, run a command, inspect an image, or edit a document.
Context	Information supplied for the current response.	Prompt, conversation, files, rules, tool output, and errors.
Skill	Reusable instructions and resources for a task class.	A testing workflow with conventions, scripts, and references.
Agent	A model operating in a loop of inspect, plan, act, observe, and revise.	Cursor Agent changes code, runs tests, reads failures, and iterates.

An agent is not automatically intelligent about the whole project. It can only make good decisions from the context it receives and the tools it is allowed to use.

Tool versus skill

A tool performs an operation. A skill explains a repeatable method for deciding which operations to perform and in what order.

For example:

- A terminal tool can execute `pytest -q`.
- A testing skill can instruct the agent to inspect the existing tests, add a regression test, run the focused test first, run the full suite second, and report failures honestly.

Agent versus ordinary chat

An ordinary chat primarily produces an answer. An agent can maintain a task loop:

1. inspect the current state;
2. formulate a plan;
3. use tools or edit files;
4. observe results such as test output;
5. revise the plan or implementation.

The loop creates more capability but also more risk. The agent may misunderstand the goal, make unnecessary changes, or act on incomplete evidence. Use approval points and review the outputs.

Context: what the AI can see

Context normally includes some combination of:

- system and application instructions;
- the current conversation;
- attached files, images, or pasted text;
- selected code, open files, project rules, and relevant repository files;
- terminal output, tests, linter messages, and tool results;
- connected application data, when the user has enabled it.

Context is finite. A long conversation does not create unlimited memory. Old details may be compressed or become less salient, while irrelevant files can consume attention. Prefer a small amount of authoritative context to a large amount of noise.

The five-part task description

For a technical task, give the model:

1. **Goal:** the required outcome.
2. **Constraints:** language, framework, compatibility, style, safety, and performance requirements.
3. **Current state:** relevant files, architecture, errors, and previous attempts.
4. **Acceptance criteria:** how completion will be checked.
5. **Permission boundary:** what may change and what must remain untouched.

For example:

```
Goal: Add CSV export to the existing dashboard.
Constraints: Keep the API stable; use TypeScript; no new dependency.
Current state: The table is in src/components/ResultsTable.tsx.
Acceptance: Export respects active filters and passes existing tests.
Permission: Inspect first; do not edit until you show a plan.
```

Context in Cursor

Cursor can use open files, selected text, the repository, terminal output, diagnostics, and explicit references. Use the `@` mechanism to point the model to a file, folder, symbol, or other supported context source.

For example, a useful request is:

```
Read @docs/csv-export.md, @src/components/ResultsTable.tsx,
and the relevant tests. Explain the current data flow and propose
the smallest implementation plan. Do not edit yet.
```

Prefer a few relevant files to the entire repository. More context is not automatically better context.

Context in ChatGPT

For recurring work, create a Project. A Project keeps related chats, uploaded reference files, and project-specific instructions together. Use one Project for a durable effort such as a course, paper, product, or research programme, and separate chats for distinct outcomes.

A useful Project instruction might be:

```
You are helping with a scientific software project.
Prefer reproducible methods and explicit assumptions.
Separate established facts from hypotheses.
When proposing code, include tests and validation steps.
Do not invent experimental results.
```

Use screenshots or uploads when the visual state matters. In **Work with Apps**, check the indicator showing which application content is being included. If it is absent, assume that the application content is not part of the prompt.

When to start a new chat

Start a new chat when:

- the objective changes;
- the conversation has accumulated many failed approaches;
- the model repeatedly forgets an important constraint;
- the task has become too broad to review;
- you can summarise the current state more clearly than the conversation does.

A good handoff summary contains the goal, decisions already made, files involved, unresolved questions, and the next action.

Rules and skills

Rules are persistent guidance, not memory of a previous conversation. In Cursor, project rules are normally stored as `.mdc` files in `.cursor/rules`; `AGENTS.md` is a simpler Markdown alternative. Use them for architecture, coding style, testing commands, naming conventions, and forbidden actions.

A good rule is short and operational:

```
description: Python project conventions
paths: **/*.py
alwaysApply: false

- Use type hints for public functions.
```

- Add or update a test for behavioural changes.
- Run `pytest -q` before claiming completion.
- Do not modify files in `data/raw/`.

Avoid placing the whole project manual in an always-on rule. Persistent context has a cost and can create contradictions. Keep universal rules short; put specialised guidance in a targeted rule or skill.

A practical skill structure

A reusable skill may contain:

```
scientific-report/  
  SKILL.md  
  references/  
    reporting-checklist.md  
  scripts/  
    validate_figures.py  
  assets/  
    report-template.tex
```

The **SKILL.md** should explain when to use the skill, the workflow, quality checks, and how to invoke its scripts. References and templates should be loaded only when relevant.

In Cursor, skills may be discovered automatically when relevant or invoked explicitly with a slash command such as `/scientific-report`. They may also be scoped to particular file patterns.

Agents and modes of work

Cursor Agent

Cursor Agent is appropriate when a task involves a real codebase and several actions. Begin with inspection and a checkpoint:

```
Inspect the repository and identify the files involved in this bug.  
Do not edit yet. Explain the likely cause, propose a minimal plan,  
and list the tests you will run. Wait for my approval.
```

After approval, let it make a small change, run focused tests, and review the diff. If it changes unrelated files, stop and discuss the scope.

ChatGPT Work and Plan

ChatGPT Work is designed for larger multi-step tasks involving files, applications, browser activity, connected tools, and finished deliverables. Plan mode is useful when you want the system to gather context and propose a sequence before execution.

The older standalone ChatGPT Agent terminology is changing: current OpenAI guidance directs users to ChatGPT Work for longer, multi-step tasks and deliverables. Workspace agents are a separate reusable option for supported Business, Enterprise, Edu, and Teachers workspaces. They can package knowledge, tools, skills, permissions, and approval points into a shared workflow.

For every agent, specify where human approval is required: sending messages, modifying shared data, deleting files, publishing, or making financial or clinical decisions.

A safe autonomy ladder

Use progressively more autonomy:

1. **Explain:** inspect and describe; do not edit.

2. **Plan:** propose files, steps, tests, and risks.
3. **Implement:** make the approved change.
4. **Verify:** run tests and report evidence.
5. **Operate:** publish, deploy, send, or modify shared systems only with explicit permission.

This is particularly important when an agent has access to a terminal, external applications, private data, or production systems.

A reliable combined workflow

1. **Frame the problem in ChatGPT.** Ask for alternatives, risks, assumptions, and a short implementation brief.
2. **Make the result durable.** Save requirements, decisions, and acceptance tests in the repository, for example as `docs/feature-brief.md`.
3. **Open the repository in Cursor.** Ask it to inspect the brief and relevant code before editing.
4. **Plan before implementation.** Require a file-level plan and identify what will not be changed.
5. **Implement incrementally.** Make one coherent change at a time and run a focused check after each change.
6. **Review the diff.** Ask Cursor to explain every changed file, or use ChatGPT for a second review.
7. **Update the durable context.** Record architectural decisions and remaining risks.

Example: fixing a bug

The login test fails with the error below. Inspect the authentication flow and the test fixture. First reproduce the failure if possible. Do not make unrelated refactors.

After identifying the cause, propose the smallest fix and a regression test. Wait for approval before editing.

After approval:

Implement only the approved fix. Run the failing test first, then the authentication test group. Show the changed files and explain any remaining warnings.

Example: adding a feature

Implement CSV export for the currently filtered results table.

Requirements:

- preserve the existing table and API;
- export the visible filtered rows;
- escape commas, quotes, and line breaks correctly;
- add tests for empty values and embedded quotes;
- do not add a dependency.

Inspect first and propose the file-level plan.

Example: research or writing

Use ChatGPT to prepare a structured brief:

```
Help me prepare a literature review on continual learning.
Separate established findings, plausible interpretations, and open
  questions.
Suggest a structure, primary sources to verify, and experiments that would
distinguish competing explanations. Do not invent citations.
```

Then use Cursor if the project includes code, data, figures, or LaTeX:

```
Read the literature brief and the existing LaTeX structure.
Add the new section without changing the citation style.
Compile the document, check for undefined references, and report warnings.
Do not claim that a result is significant unless the analysis supports it.
```

Prompt patterns

Understand before changing

```
Read the relevant files and tests. Explain the current data flow,
the likely source of the problem, and two possible fixes. Do not edit.
End with the smallest safe plan.
```

Implement with a boundary

```
Implement the approved plan. Change only the necessary files.
Preserve the public API and existing style. Add regression tests.
Run focused tests, report failures honestly, and show the diff summary.
```

Debug from evidence

```
Here is the failing test and the complete error output.
First reproduce or inspect the failure. Identify the violated assumption.
Propose a minimal fix, then test the fix against the failing case
and the nearest relevant edge cases.
```

Review

```
Review this change as a cautious maintainer. Look for correctness bugs,
missing tests, security problems, performance regressions, and scope creep
.
Rank findings by severity and cite the file and relevant code.
```

Common mistakes and safety

- Asking for the whole application to be rewritten before understanding the architecture.
- Giving contradictory instructions in prompts, rules, and project documentation.
- Treating a fluent answer as evidence that code was executed or tested.
- Allowing an agent to edit many files without a Git checkpoint.

- Including secrets, API keys, private patient data, or unnecessary personal information in context.
- Installing skills, extensions, or integrations without checking their permissions.
- Accepting generated code without reading the diff and running independent checks.
- Allowing a model to publish, send, delete, or deploy without a clear approval boundary.

One-minute checklist

Before asking:

- What is the outcome?
- Which files or sources are authoritative?
- What must not change?
- What counts as success?
- Does the model need a tool, a skill, or simply a clearer explanation?

Before accepting:

- Did it inspect first?
- Did it use evidence?
- Did it stay within scope?
- Did tests or independent checks support the result?
- Can you explain the result?
- Is there a recoverable version?

Further reading

References

- [1] OpenAI, Work with Apps on macOS, <https://help.openai.com/en/articles/10119604-work-with-apps-on-macos>.
- [2] OpenAI, Projects in ChatGPT, <https://help.openai.com/en/articles/10169521-using-projects-in-chatgpt>.
- [3] OpenAI, ChatGPT Work, <https://chatgpt.com/work/>.
- [4] OpenAI, Introducing workspace agents in ChatGPT, <https://openai.com/index/introducing-workspace-agents-in-chatgpt/>.
- [5] Cursor, Rules, <https://cursor.com/docs/rules>.
- [6] Cursor, Agent Skills, <https://cursor.com/docs/skills>.
- [7] Cursor, Context, <https://cursor.com/learn/context>.